Server Framing Protocol Specification

Introduction

This document defines the communication protocol utilized by backend services such as:

- SHELDataEngine
- JournalWriter
- JournalReader

This protocol is independent of application specific data or message protocol it envelops.

Client to Server Protocol Specification

- The client to server communication is done over a single stream of JSON formatted request objects
- The request objects are serialized to a single line of text and delimited by newline (i.e. NDJSON/JSONLines)
- Each request object must contains an integer field named “id” which must be unique for the TCP session. The rest of the fields are application specific.

Server to Client Framing Protocol Specification

- *Stream*: A unidirectional flow of bytes from server to client, which may carry one or more messages.
- *Message*: A logical communication unit defined in an external protocol. It is application and possibly request specific, and the framing protocol described here is agnostic to the messaging protocol it carries.
- *Frame*: The smallest unit of communication, each frame carries a header, which at minimum identifies the stream (and consequently the request) to which the frame belongs.
- The client to server communication is done over a single TCP connection which carries a number of streams.

- The response to each request forms a separate stream, uniquely identified by the Request ID.
- Frames from different streams are interleaved (multiplexed) and reassembled by the client via the Request ID on each frame. As a result requests do not block each other and long running e.g. live requests can exist on the same TCP connection.
- Integer fields are binary formatted, big-endian numbers.

Frame Header

Field Name	Length	Type	Description
Request ID	4	Integer	Identifies the request
Payload Size	4	Integer	Size of payload in bytes.
Flags	1	Bitmask	See ‘Flags’ table.
Status Code	1	Integer	If ‘End of request’ flag is not set value is 0. See ‘Status Code’ table.
Uncompressed Size	0 or 4	Integer	*Optional*. Only present if the ‘Compressed’ flag is set.

- All integer fields are encoded in Big Endian format.

Flags

Bit	Description
0x01	End of request. Indicates the end of the reply for the particular request id.
0x02	Server response. The attached payload is an out of band message generated by the server.
0x04	Compressed. The payload is compressed. The ‘Uncompressed Size’ field is present to aid memory allocation. Note: The compression protocol utilized is not explicitly defined. The client must make assumptions. This is to be tackled as a future improvement.

Status Code

Code	Description
0	No error.
1	Resource not found
2	Other Error

Request Lifetime

- A historical / closed-ended request sets the `"End of request"` flag to signify completion. After that the stream is closed and no further frames will be sent for that Request ID.
- A live / open-ended request simply does not close the stream by sending a frame with `"End of request"` flag set. As such it acts as a subscription to live data.
- The client can terminate any active request, whether open or closed ended at any point by sending a `"cancel"` request:

```
{
  "id":                <integer>,
  "action":            "cancel",
  "target-request-id": <integer>
}
```

  The response stream to the cancel request always returns with no error, even if the target request completed naturally before the cancel request could be processed.
- If the target request is terminated due to a cancel request, it will send a final frame close its stream by setting the “End of Request” flag. Additionally the “Status Code” will be set to 2 (Other Error) with the body of the error containing type `"CANCELED"`.

Out Of Band Messages

Control Messages

- These messages can appear mid stream and their meaning is defined at the application level. For example the following control message indicates the point where the stream has switched from rewind to live data.

```
{
  "category": "control",
  "type":     <string>,  //"end-of-rewind", ...
}
```

Error Messages

- The value of `Status Code` can only be inspected when `End of Request` flag is set. If an error code is being reported, the server can send additional information about the error. This

data is considered “out-of-band”, meaning it is not part of the application protocol. The `Server Response` flag is set to indicate this. The format of this data is the following:

```
{
  "category": "error",
  "type":     <string>,  //"RESOURCE_NOT_FOUND", "VALIDATION_ERROR", "RATE_LIMIT_ERROR", "INTERNAL_ERROR", "CANCELED", "UNKNOWN_ERROR", "SERVICE_UNAVAILABLE"
  "message":  <string>   //A free-form error message
}

```

- Not all possible errors can be enumerated in the `Status Code` field of the header, although some like `"CANCELED"` seem like good candidates to be added because they concern the framing (request / response) protocol itself and not any application specific conditions.